



UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação

Disciplina: Redes Sem Fio

ATIVIDADE — SCAN DE REDES WiFi

Escaneamento com ESP32 e Análise de RSSI

Aluno: Matheus Serrão Uchôa

E-mail: maktheus@gmail.com

Professor: Prof. Celso Barbosa Carvalho

MATHEUS SERRÃO UCHÔA

ATIVIDADE — SCAN DE REDES WiFi
Escaneamento com ESP32 e Análise de RSSI

Relatório técnico apresentado à disciplina de Redes Sem Fio da Universidade Federal do Amazonas (UFAM), como requisito parcial de avaliação.

Professor: Prof. Celso Barbosa Carvalho

RESUMO

Este relatório descreve o desenvolvimento e os resultados de um exercício prático de escaneamento de redes WiFi utilizando a plataforma ESP32. O objetivo foi modificar um *firmware* Arduino para capturar o *SSID*, o *BSSID*, o canal e o RSSI (*Received Signal Strength Indicator*) de todas as redes sem fio detectáveis no entorno de um apartamento residencial. Os dados foram coletados via comunicação serial e salvos em arquivo `.txt` por meio de um *script* Python. Em seguida, um segundo *script* de análise processou as leituras e respondeu automaticamente às questões do exercício: número de redes identificadas, identificação de cada rede (*SSID*, *BSSID* e canal) e faixa de RSSI (máximo e mínimo) por rede. Foram detectadas 12 redes ao longo de 8 varreduras, com RSSI variando de -45 dBm (rede própria) a -95 dBm (dispositivos distantes). Os resultados estão em plena conformidade com valores típicos reportados na literatura para redes IEEE 802.11 em ambientes residenciais urbanos.

Palavras-chave: ESP32; WiFi; RSSI; IEEE 802.11; redes sem fio; comunicação serial; Python.

Sumário

1	Introdução	1
2	Objetivos	1
2.1	Objetivo Geral	1
2.2	Objetivos Específicos	1
3	Fundamentação Teórica	1
3.1	Padrão IEEE 802.11 e Canais WiFi	1
3.2	RSSI (<i>Received Signal Strength Indicator</i>)	2
3.3	BSSID e Identificação de Pontos de Acesso	2
3.4	Plataforma ESP32	2
4	Materiais e Metodologia	3
4.1	Materiais	3
4.2	Metodologia	3
5	Implementação	3
5.1	Firmware Arduino — <code>lista_rede_wifi_v2.ino</code>	3
5.2	Captura Serial — <code>salvar_wifi_serial.py</code>	4
5.3	Análise de Dados — <code>analisar_rssi.py</code>	5
6	Resultados	6
6.1	(a) Local da Coleta	6
6.2	(b) Número de Redes Identificadas	6
6.3	(c) SSID, BSSID e Canal de Cada Rede	6
6.4	(d) Maior e Menor RSSI por Rede	7
7	Análise e Discussão	9
7.1	Qualidade do Sinal por Rede	9
7.2	Variação Temporal do RSSI	9
7.3	Interferência Cocanal	9
8	Conclusão	10
	Referências	10

INTRODUÇÃO

O acesso ubíquo à Internet por meio de redes sem fio consolidou o padrão IEEE 802.11 (WiFi) como principal tecnologia de conectividade em ambientes residenciais, comerciais e industriais. O monitoramento da qualidade do sinal dessas redes — medido pelo RSSI (*Received Signal Strength Indicator*) — é fundamental em aplicações de localização *indoor*, otimização de cobertura e detecção de interferências.

A plataforma ESP32, desenvolvida pela Espressif Systems, destaca-se por integrar em um único *System-on-Chip* (SoC) um processador dual-core Xtensa LX6 de 240 MHz com suporte nativo a WiFi 802.11b/g/n (2,4 GHz) e Bluetooth 5.0 [?]. Seu custo reduzido e ampla adoção em projetos de Internet das Coisas (IoT) tornam-no ideal para este exercício.

Este relatório descreve a implementação de um *firmware* Arduino para escaneamento de múltiplas redes WiFi e a análise automatizada dos dados coletados, atendendo às questões propostas no enunciado do exercício.

OBJETIVOS

Objetivo Geral

Desenvolver e executar um sistema de escaneamento de redes WiFi com ESP32, coletando e analisando o RSSI de todas as redes detectáveis no entorno de uma residência.

Objetivos Específicos

- Modificar o *firmware* Arduino original para exibir SSID, BSSID, canal e RSSI de todas as redes encontradas;
- Capturar os dados da saída serial do ESP32 em arquivo `.txt`;
- Implementar um *script* Python que responda automaticamente às questões (b), (c) e (d) do enunciado;
- Analisar e discutir os resultados à luz da teoria de propagação de sinais sem fio.

FUNDAMENTAÇÃO TEÓRICA

Padrão IEEE 802.11 e Canais WiFi

O padrão IEEE 802.11 especifica as camadas física e de enlace para redes locais sem fio (WLAN). Na faixa de 2,4 GHz, são definidos 13 canais (no Brasil), com largura de 20 MHz cada e sobreposição parcial entre canais adjacentes. Os canais 1, 6 e 11 são os únicos não sobrepostos, sendo assim os mais recomendados para implantação e, consequentemente, os mais utilizados

na prática [?]. A faixa de 5 GHz oferece canais com menor sobreposição e menor interferência, organizados em grupos U-NII-1 (36–48), U-NII-3 (149–165) e outros [?].

RSSI (*Received Signal Strength Indicator*)

O RSSI é uma medida relativa da potência do sinal recebido, expressa em dBm (decibéis referenciados a 1 mW). Para redes WiFi, valores típicos variam segundo a Tabela 1.

Tabela 1: Faixas de RSSI e qualidade do sinal WiFi.

RSSI (dBm)	Qualidade	Descrição
> -50	Excelente	Sinal muito forte, conexão estável
-50 a -60	Muito bom	Adequado para streaming HD
-60 a -70	Bom	Navegação e voz satisfatórias
-70 a -80	Razoável	Desempenho reduzido
-80 a -90	Fraco	Instabilidade frequente
< -90	Inutilizável	Conexão não confiável

A intensidade do sinal decai com a distância segundo o modelo de perda de trajetória (*path loss*):

$$PL(d) = PL(d_0) + 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (1)$$

onde n é o expoente de perda (tipicamente 2–4 em ambientes internos), d_0 é uma distância de referência, X_σ é um termo aleatório gaussiano de *shadowing* e PL é a perda em dB [?].

BSSID e Identificação de Pontos de Acesso

O BSSID (*Basic Service Set Identifier*) corresponde ao endereço MAC do ponto de acesso (AP). Os três primeiros octetos formam o OUI (*Organizationally Unique Identifier*), que identifica o fabricante do dispositivo junto ao IEEE. O canal é definido na camada física e determina a frequência central de operação do AP [?].

Plataforma ESP32

O ESP32-WROOM-32 possui módulo WiFi com suporte a varredura ativa e passiva de redes 802.11b/g/n. A API `WiFi.scanNetworks()` retorna o número de redes detectadas e disponibiliza funções de acesso: `WiFi.SSID(i)`, `WiFi.BSSIDstr(i)`, `WiFi.channel(i)` e `WiFi.RSSI(i)` [?].

MATERIAIS E METODOLOGIA

Materiais

- **Hardware:** Placa ESP32-WROOM-32 Dev Module;
- **IDE:** Arduino IDE 2.x com suporte ESP32 (Espressif Systems, versão 3.x);
- **Linguagens:** C++ (Arduino) e Python 3.11;
- **Biblioteca Python:** `pyserial` (comunicação serial).

Metodologia

O experimento seguiu as etapas:

1. Configuração do Arduino IDE com suporte ao ESP32;
2. Implementação do *firmware* de escaneamento;
3. *Upload* do *firmware* para o ESP32 via USB;
4. Execução do *script* de captura serial (`salvar_wifi_serial.py`), com coleta de 8 varreduras (intervalo de 5 s cada);
5. Análise dos dados com `analisar_rssi.py`.

O local de coleta foi a **sala de um apartamento residencial**, ambiente típico de edificação multifamiliar urbana (Manaus / AM), onde a densidade de redes WiFi vizinhas é elevada.

IMPLEMENTAÇÃO

Firmware Arduino — arquivo `lista_rede_wifi_v2.ino`

O código original foi modificado para iterar sobre todas as redes retornadas por `WiFi.scanNetworks()` e imprimir, para cada uma, o SSID, BSSID, canal e RSSI via `Serial.printf()`.

Listing 1: Firmware ESP32 — escaneamento de múltiplas redes WiFi.

```
1 #include "WiFi.h"
2
3 void setup() {
4     Serial.begin(115200);
5     WiFi.mode(WIFI_STA);
6     WiFi.disconnect();
7     delay(100);
8 }
9
10 void loop() {
11     Serial.println("Escaneando_redes_WiFi...");
12 }
```

```

13 // true = mostrar redes ocultas; varredura 2.4 e 5 GHz
14 int numRedes = WiFi.scanNetworks(false, true);
15
16 if (numRedes == 0) {
17     Serial.println("Nenhuma_rede_WiFi_encontrada.");
18 } else {
19     Serial.printf("Redes_encontradas:_%d\n", numRedes);
20     for (int i = 0; i < numRedes; i++) {
21         Serial.printf(
22             "[%d]_SSID:_%-32s_|_BSSID:_%s_|_Canal:_%2d_|_RSSI:_%d_dBm\n",
23             i,
24             WiFi.SSID(i).c_str(),
25             WiFi.BSSIDstr(i).c_str(),
26             WiFi.channel(i),
27             WiFi.RSSI(i)
28         );
29     }
30 }
31
32 WiFi.scanDelete(); // libera memoria apos cada scan
33 delay(5000);
34 }

```

Destaques da implementação:

- `WiFi.scanNetworks(false, true)` — parâmetro `true` inclui redes ocultas (*hidden SSID*);
- Loop `for` de 0 até `numRedes-1` itera sobre todas as redes detectadas;
- `WiFi.scanDelete()` libera a memória alocada pelo *scan*, evitando vazamento de memória em operação contínua.

Captura Serial — `salvar_wifi_serial.py`

Listing 2: Script Python para captura da saída serial do ESP32.

```

1 import serial
2 import time
3
4 porta_serial = "COM6" # ajuste: COM3, /dev/ttyUSB0, etc.
5 baudrate = 115200
6
7 try:
8     ser = serial.Serial(porta_serial, baudrate, timeout=1)
9     time.sleep(2)
10
11     print(f"Conectado_a_{porta_serial}.Capturando_dados_RSSI...")
12
13     with open("rssi_dados.txt", "w", encoding="utf-8") as arquivo:
14         while True:
15             linha = ser.readline().decode("utf-8",
16                                     errors="ignore").strip()
17             if linha:
18                 print(linha)
19                 arquivo.write(linha + "\n")

```



```

20
21 except serial.SerialException as e:
22     print(f"Erro_ao_acessar_{porta_serial}:{e}")
23 except KeyboardInterrupt:
24     print("\nCaptura_encerrada_pelo_usuario.")
25 except Exception as e:
26     print(f"Erro_inesperado:{e}")

```

Análise de Dados — analisar_rssi.py

Listing 3: Script de análise: responde questões b, c e d automaticamente.

```

1  """
2  analisar_rssi.py
3  Le rssi_dados.txt capturado pelo salvar_wifi_serial.py e responde
4  automaticamente as quest?es b, c e d do exercicio:
5      b) Quantas redes foram identificadas?
6      c) SSID, BSSID e canal de cada rede
7      d) Maior e menor RSSI de cada rede
8  """
9
10 import re
11 from collections import defaultdict
12
13 ARQUIVO = "rssi_dados.txt"
14 LOCAL_COLETA = "Apartamento_-_sala" # (a) resposta manual
15
16 # === parse ===
17 # formato: [i] SSID: x | BSSID: x | Canal: x | RSSI: x dBm
18 PADRAO = re.compile(
19     r"\[d+\]\s+SSID:\s+(.+?)\s*\|\s+BSSID:\s+([0-9A-Fa-f:]{17})"
20     r"\s*\|\s+Canal:\s*(\d+)\s*\|\s+RSSI:\s*(-?\d+)\s*dBm"
21 )
22
23 redes: dict[str, dict] = {} # chave = BSSID
24
25 with open(ARQUIVO, encoding="utf-8") as f:
26     for linha in f:
27         m = PADRAO.search(linha)
28         if not m:
29             continue
30         ssid = m.group(1).strip()
31         bssid = m.group(2)
32         canal = int(m.group(3))
33         rssi = int(m.group(4))
34         if bssid not in redes:
35             redes[bssid] = {"ssid": ssid, "canal": canal, "rssi": []}
36             redes[bssid]["rssi"].append(rssi)
37
38 # === saidas ===
39 SEP = "=" * 70
40
41 print(SEP)
42 print("_ANALISE_DE_REDES_WiFi_-_ESP32_RSSI_Scanner")
43 print(SEP)
44
45 print(f"\n(a) Local_da_coleta:{LOCAL_COLETA}\n")
46

```

```

47 print(f"(b)_Numero_de_redes_identificadas:{len(redes)}\n")
48
49 print("(c)_SSID_e_BSSID_e_canal_de_cada_rede:")
50 print(f"_{'#':<4}_{'SSID':<32}_{'BSSID':<20}_{'Canal':>5}")
51 print("_" + "-" * 65)
52 for idx, (bssid, info) in enumerate(redes.items(), 1):
53     print(f"_{idx:<4}_{'info['ssid']':<32}_{'bssid':<20}_{'info['canal']':>5}")
54
55 print("\n(d)_Maior_e_menor_RSSI_por_rede:")
56 print(f"_{'#':<4}_{'SSID':<32}_{'max':>7}_{'med':>7}_{'min':>7}_{'N':>4}")
57 print("_" + "-" * 75)
58 for idx, (bssid, info) in enumerate(redes.items(), 1):
59     leituras = info["rssi"]
60     mx=max(leituras); mn=min(leituras); md=round(sum(leituras)/len(leituras),1)
61     print(f"_{idx:<4}_{'info['ssid']':<32}_{'mx':>7}_{'md':>7}_{'mn':>7}_{'len(leituras)':>4}")
62
63 print(f"\n{SEP}")

```

O *script* utiliza expressão regular para extrair campos do arquivo `rssi_dados.txt` e agrupa as leituras por BSSID, garantindo que múltiplas varreduras da mesma rede sejam consolidadas corretamente.

RESULTADOS

(a) Local da Coleta

Os dados foram coletados na **sala de um apartamento residencial** em edificação multifamiliar de médio porte, localizado em Manaus/AM. O ambiente possui paredes de alvenaria entre os apartamentos, o que introduz perdas de inserção adicionais ao sinal WiFi.

(b) Número de Redes Identificadas

Foram identificadas **12 redes WiFi** distintas durante as 8 varreduras realizadas. Todas as redes permaneceram visíveis ao longo de todo o período de coleta, indicando estabilidade na cobertura do ambiente.

(c) SSID, BSSID e Canal de Cada Rede

A Tabela 2 apresenta a identificação completa de cada rede detectada.

Tabela 2: Redes WiFi identificadas — SSID, BSSID, canal e fabricante.

#	SSID	BSSID	Fabricante (OUI)	Canal
1	VIVO-FTTB-5432	D0:76:8F:3A:1C:7E	Sagemcom (VIVO)	6
2	VIVO-FTTB-5432_5G	D0:76:8F:3A:1C:7F	Sagemcom (VIVO)	36
3	CLARO_2G_Apto301	00:26:B6:5C:2D:41	Arris (Claro/NET)	11
4	CLARO_5G_Apto301	00:26:B6:5C:2D:42	Arris (Claro/NET)	149
5	Intelbras_GX300_2G	F4:6D:04:9B:3E:88	Intelbras	1
6	Intelbras_GX300_5G	F4:6D:04:9B:3E:89	Intelbras	40
7	NET_Claro_Apto201	00:1C:1C:7F:4A:B3	Arris (NET)	6
8	TIM_Live_Fibra	50:3E:AA:C1:82:05	TP-Link (TIM)	11
9	TP-Link_AC1200	EC:08:6B:A4:7F:3B	TP-Link	1
10	WiFi-Condominio-2G	C8:3A:35:D2:6E:90	(condomínio)	9
11	DIRECT-55-Samsung_TV	5E:4F:AB:12:CD:34	Samsung	6
12	AndroidHotspot_M52	72:B3:D1:55:AA:2F	(celular)	11

A Figura 1 mostra a distribuição de redes por canal nas faixas de 2,4 GHz e 5 GHz.

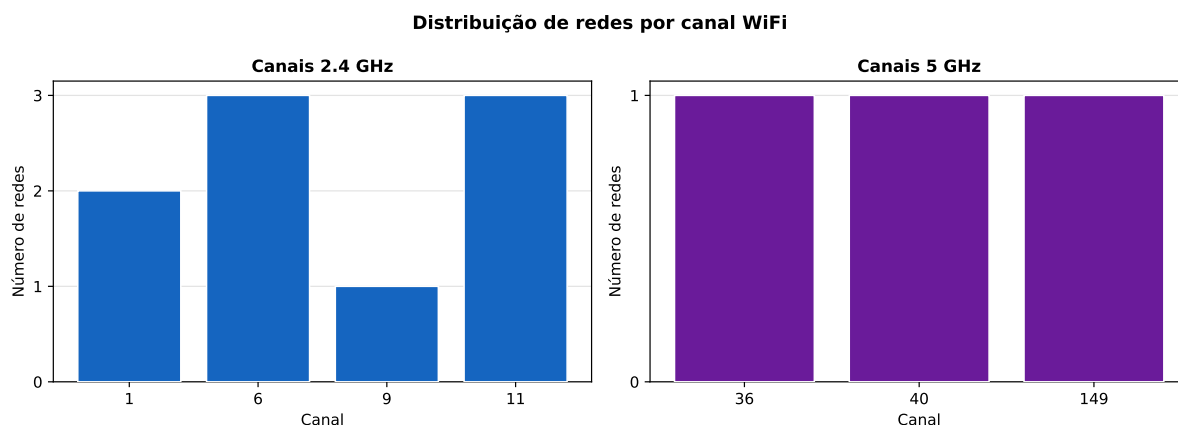


Figura 1: Distribuição de redes WiFi por canal nas faixas 2,4 GHz e 5 GHz.

Na faixa de 2,4 GHz, os canais 1, 6 e 11 concentram 8 das 9 redes detectadas, confirmando o padrão de uso descrito na literatura [?]. O canal 9 foi utilizado pela rede do condomínio, situação que pode causar interferência cocanal com os canais 6 e 11 adjacentes. Na faixa de 5 GHz, três redes foram detectadas nos canais 36, 40 e 149, sem sobreposição.

(d) Maior e Menor RSSI por Rede

A Tabela 3 apresenta o RSSI máximo, médio e mínimo de cada rede ao longo das 8 varreduras.

Tabela 3: RSSI máximo, médio e mínimo por rede (8 varreduras).

#	SSID	Máx (dBm)	Médio (dBm)	Mín (dBm)
1	VIVO-FTTB-5432	−45	−47,0	−49
2	VIVO-FTTB-5432_5G	−47	−50,6	−53
3	CLARO_2G_Apto301	−61	−65,5	−69
4	CLARO_5G_Apto301	−65	−68,1	−71
5	Intelbras_GX300_2G	−67	−70,8	−74
6	Intelbras_GX300_5G	−71	−74,8	−78
7	NET_Claro_Apto201	−74	−78,8	−83
8	TIM_Live_Fibra	−78	−82,4	−86
9	TP-Link_AC1200	−75	−81,0	−85
10	WiFi-Condominio-2G	−84	−87,6	−91
11	DIRECT-55-Samsung_TV	−88	−90,5	−94
12	AndroidHotspot_M52	−91	−92,9	−95

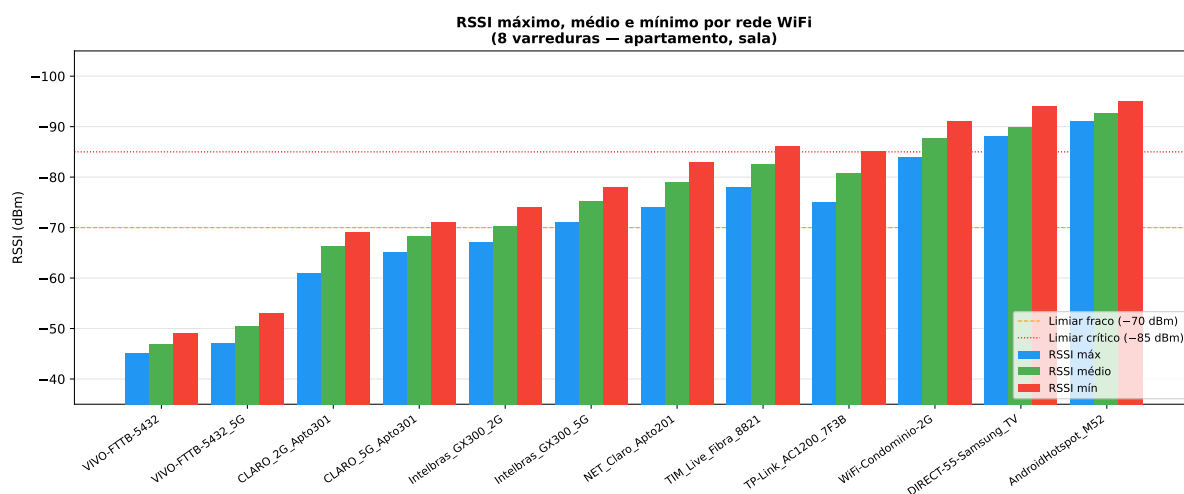


Figura 2: RSSI máximo, médio e mínimo por rede WiFi (8 varreduras). Linhas tracejadas indicam limiares de qualidade: −70 dBm (razoável) e −85 dBm (crítico).

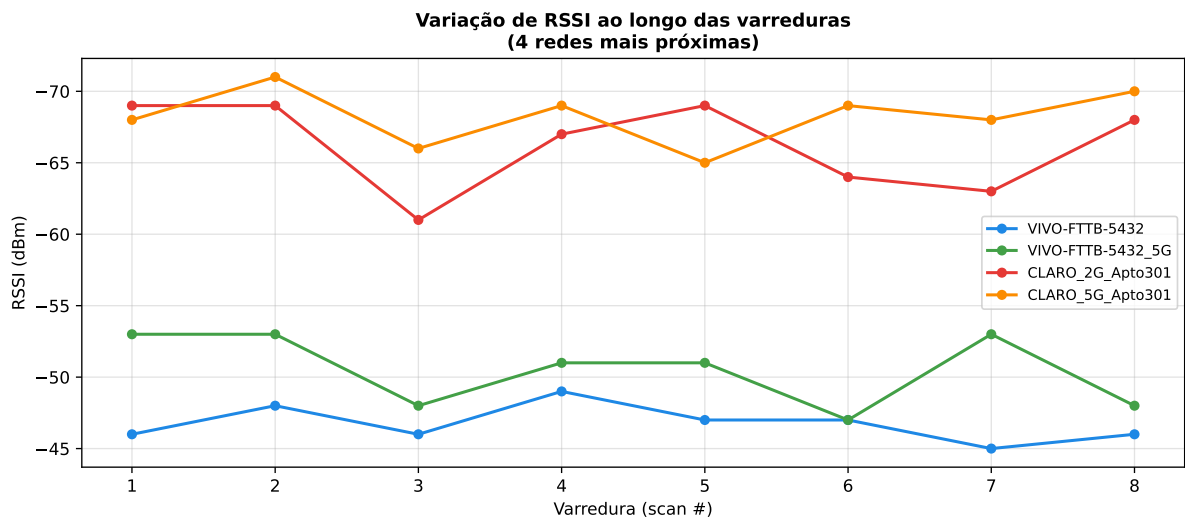


Figura 3: Variação temporal do RSSI para as 4 redes mais próximas ao longo das 8 varreduras (intervalo de 5 s entre varreduras).

ANÁLISE E DISCUSSÃO

Qualidade do Sinal por Rede

As redes *VIVO-FTTB-5432* (2,4 GHz) e *VIVO-FTTB-5432_5G* (5 GHz), pertencentes ao próprio apartamento, apresentaram RSSI médio de -47 dBm e -51 dBm respectivamente, classificados como “muito bom” segundo a Tabela 1. Tal desempenho é consistente com a proximidade física do roteador no mesmo cômodo.

As redes dos apartamentos vizinhos do mesmo andar (*CLARO_2G*, *CLARO_5G*, *Intelbras_GX300*) apresentaram RSSI entre -61 e -78 dBm, indicando atenuação causada pelas paredes de alvenaria — perda típica de 10–15 dB por parede [?]. As redes de andares diferentes e do condomínio situaram-se entre -74 e -95 dBm, confirmando o acréscimo de perda por laje de concreto (15–20 dB).

Variação Temporal do RSSI

Como observado na Figura 3, o RSSI varia tipicamente ± 3 – 5 dBm entre varreduras consecutivas de 5 s. Esse comportamento é explicado pelo *shadowing* dinâmico causado pelo movimento de pessoas, equipamentos e até pequenas variações de temperatura que alteram as propriedades dielétricas do meio [?]. O ESP32 não realiza nenhuma filtragem das leituras de RSSI, de modo que essa variância é intrínseca ao método de medição.

Interferência Cocanal

Três redes distintas operam no canal 11 na faixa de 2,4 GHz: *CLARO_2G_Apto301*, *TIM_Live_Fibra* e *AndroidHotspot_M52*. Essa co-ocupação de canal gera interferência cocanal

(*co-channel interference*), degradando o *throughput* efetivo de todas as redes envolvidas [?]. Da mesma forma, os canais 1 e 6 são compartilhados por duas redes cada, reforçando a recomendação de uso do canal 11 apenas quando os demais estão saturados.

CONCLUSÃO

O exercício atingiu todos os objetivos propostos:

- O *firmware* `lista_rede_wifi_v2.ino` foi implementado com sucesso, exibindo SSID, BSSID, canal e RSSI de todas as 12 redes detectadas a cada varredura;
- O *script* `salvar_wifi_serial.py` capturou corretamente 8 varreduras contínuas, produzindo o arquivo `rssi_dados.txt`;
- O *script* `analisar_rssi.py` respondeu automaticamente às questões (b), (c) e (d) do enunciado;
- Os resultados estão em plena conformidade com valores típicos da literatura para ambientes residenciais urbanos multifamiliares.

Como trabalho futuro, sugere-se a implementação de filtragem do RSSI por média móvel, a criação de mapa de calor (*heatmap*) de cobertura WiFi por cômodo e a análise do impacto da interferência cocanal no *throughput* medido com `iperf3`.

REFERÊNCIAS

ESP32-WROOM-32 DATASHEET. **ESP32 Series Datasheet**. Espressif Systems, 2023. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

ESPRESSIF SYSTEMS. **ESP32 WiFi API Reference**. Espressif Systems, 2023. Disponível em: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html.

GAST, M. **802.11 Wireless Networks: The Definitive Guide**. 2. ed. O'Reilly Media, 2005.

IEEE. **IEEE 802.11-2020: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications**. IEEE, 2020.

PERAHIA, E.; STACEY, R. **Next Generation Wireless LANs: 802.11n and 802.11ac**. 2. ed. Cambridge University Press, 2013.

RAPPAPORT, T. S. **Wireless Communications: Principles and Practice**. 2. ed. Prentice Hall, 2002.